

DriftingVLA: Native One-Step Vision-Language-Action Generation via Per-Dimension Temporal Drifting

Yuxuan Gao^{1,*}, Shiqi Zhang^{1,*}, Yedong Shen¹, Yifan Duan², Wenhao Yu¹, Xin Zhang¹, Siyuan Cao¹, Jiajun Deng¹, Yanyong Zhang^{1,†} *Fellow, IEEE*

Abstract—Conventional flow-based vision-language-action (VLA) models support expressive continuous action generation but rely on multi-step refinement to produce each action chunk, increasing latency in online robot control. To address this issue, we introduce DriftingVLA, a native one-step VLA that generates a complete action chunk with a single action-expert forward pass. Rather than learning a flow field that requires iterative integration at inference, DriftingVLA uses a distribution-drifting objective to learn a direct noise-to-action-chunk mapping for one-step deployment. Since robot action dimensions carry distinct control semantics and distributional characteristics, we further introduce Per-Dimension Temporal Drifting (PDTD). PDTD treats the complete temporal trajectory of each action dimension as a separate drifting unit, enabling finer-grained modeling and shaping of dimension-specific action distributions. This per-dimension decomposition applies only to the training objective; the shared VLA model still generates the complete action chunk jointly, thereby preserving cross-dimensional dependencies. DriftingVLA achieves 98.32% success on LIBERO, 81.09% on RoboTwin 2.0, and 77.67% across six real-world single- and dual-arm tasks, outperforming the evaluated multi-step flow policy and one-step VLA baselines. Native one-step deployment also accelerates action-chunk generation by 3.36 $\times$, eliminating iterative refinement without sacrificing control performance.

Index Terms—Robot learning, robotic manipulation, vision-language-action models, one-step action generation, distribution drifting, flow matching.

I. INTRODUCTION

Vision-language-action (VLA) models have emerged as a promising paradigm for general-purpose robotic manipulation by combining pretrained multimodal representations with end-to-end continuous control [1]–[6]. Recent models further improve action expressiveness by pairing a large vision-language backbone with a generative continuous-action head [7]–[11]. In particular, π_0 [12] and $\pi_{0.5}$ [13] use flow matching to generate action chunks, enabling the policy to model complex and multimodal robot behaviors while retaining the semantic and perceptual priors of a pretrained VLM backbone. This design, however, introduces a deployment-time cost: each action chunk is recovered by numerically integrating a learned vector field, requiring multiple sequential evaluations of the action expert. Although the multimodal prefix can be cached across the integration trajectory, the repeated action-expert computation remains directly on the robot-time critical path.

This latency has motivated a growing body of work on faster generative action policies [10], [14]–[17]. Some approaches reduce execution latency while retaining an iterative generator, for example by overlapping action generation with execution or prioritizing near-term actions during flow sampling [18]–[20]. More directly, recent one-step VLA methods modify the training or action-generation formulation [17], [21]–[23]. SnapFlow [24] progressively distills a multi-step flow trajectory into a shortcut predictor, while MeanFlowVLA [25] adapts the MeanFlow formulation [26] to predict an interval-averaged velocity suitable for one-step action generation. These methods demonstrate that flow-based VLAs can be accelerated substantially, but their one-step generators remain derived from the transport process inherited from flow matching. This motivates a different formulation: adapting a pretrained flow-based VLA into a native one-step conditional generator without relying on an inference-time transport trajectory.

Distribution drifting provides a natural foundation for this formulation. Drifting Models [27] move distribution refinement from inference-time sample evolution to optimization-time generator learning. Rather than learning a local denoising or transport field that must be repeatedly evaluated after deployment, a drifting field specifies distribution-level corrections during training, and these corrections are progressively absorbed into the generator parameters. Drift-Based Policy Optimization (DBPO) [28] introduced a native one-step policy learning framework for robotic control. Its generative component, Drift-Based Policy (DBP), applies distribution drifting in action space using multiple sibling action hypotheses during training while retaining single-sample, one-evaluation deployment. Extending DBP to pretrained flow-based VLAs, however, is not a direct substitution and raises two coupled design challenges. First, the generative interface must be adapted: the action expert of a pretrained flow VLA is specialized for local velocity prediction, whereas native drifting requires a direct noise-to-action generator. Second, the distributional geometry must be reconsidered: a VLA predicts a structured, temporally extended action chunk, so the organization of its temporal and action dimensions directly determines the geometry used to construct drifting updates.

Specifically, an action chunk is naturally represented as a tensor $\mathbf{A} \in \mathbb{R}^{H \times D}$, where the temporal horizon H and action dimension D form two semantically different axes [7], [29]–[33]. The Chunk-wise and Step-wise drifting organizations considered in DBP expose complementary limitations when applied to this structure. Chunk-wise drifting flattens the entire $H \times D$ tensor into one group. It preserves the complete temporal horizon, but translation, rotation, gripper, and other

*These authors contributed equally.

[†]Corresponding author.

¹University of Science and Technology of China, Hefei 230026, China. Emails: {yuxuangao, zhangshiqi_1127, sydong2002, wenhaoyu, xzhang12, caosiyuan}@mail.ustc.edu.cn, {dengjj, yanyongz}@ustc.edu.cn

²LYNSENSE, Shanghai, China. Email: duanyifan@lynsense.net

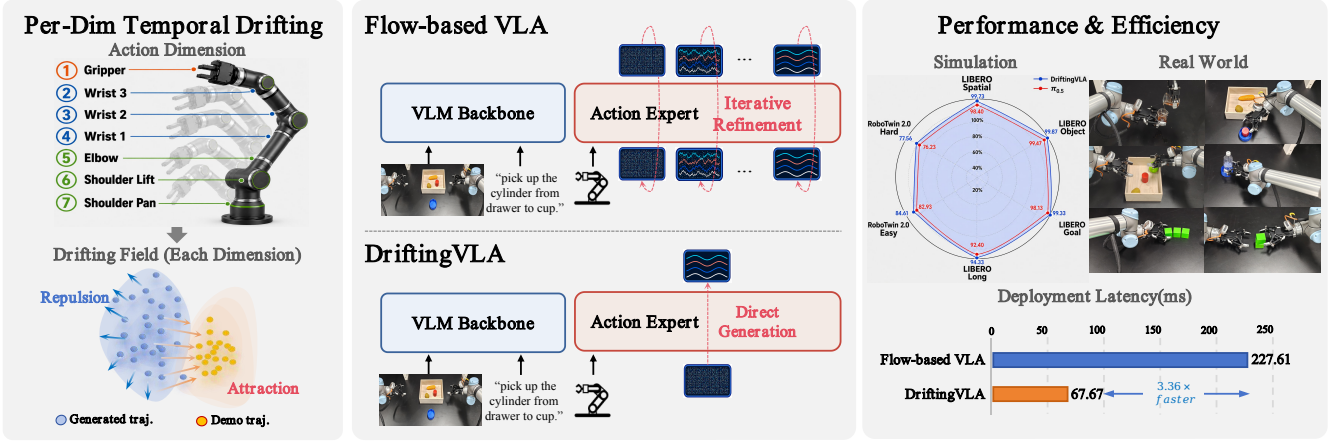


Fig. 1. Overview of DriftingVLA. Conventional flow-based VLAs generate an action chunk through iterative action-expert evaluations along a numerical integration trajectory. DriftingVLA instead performs distribution refinement during training via Per-Dimension Temporal Drifting (PDTD) and deploys a direct conditional generator that requires only one action-expert evaluation per action chunk. The right panel summarizes simulation and real-world performance together with deployment-time efficiency.

heterogeneous action channels must share the same distance scale, neighborhood statistics, and drifting geometry. Step-wise drifting instead treats each temporal slice independently. This reduces the scope of each group, but fragments the complete temporal evolution of every action channel across independently normalized problems while still mixing heterogeneous channels at each timestep. In short, the former preserves temporal completeness but mixes action-channel geometry, whereas the latter reduces group dimensionality but breaks each channel’s temporal trajectory. These limitations motivate the remaining natural orientation: organizing drifting along the complete temporal trajectory of each individual action channel.

To address these two challenges, we introduce DriftingVLA, a native one-step VLA that combines pretrained multimodal representations with direct distribution-drifting action generation. Figure 1 summarizes the shift from iterative robot-time refinement to training-time distribution drifting and direct one-step deployment. For the generative interface, DriftingVLA retains the pretrained multimodal backbone, freshly initializes the continuous action-generation branch, and jointly post-trains both components so that the action expert learns a direct mapping from action-shaped Gaussian noise to a complete action chunk rather than inheriting a flow-velocity parameterization. For the structured action geometry, we propose Per-Dimension Temporal Drifting (PDTD), which treats each channel trajectory $\mathbf{A}_{:,d} \in \mathbb{R}^H$ as one drifting group and defines its distributional geometry independently of heterogeneous action dimensions. Crucially, PDTD factorizes only the geometry used to construct the training signal, not the action generator itself. The shared VLM and action Transformer still jointly generate the complete action chunk and retain the capacity to model cross-dimensional dependencies required for coordinated control. During training, the multimodal prefix is additionally shared across sibling samples so that distribution learning does not require repeated VLM computation.

We evaluate DriftingVLA on LIBERO, RoboTwin 2.0,

and six real-world manipulation tasks spanning both single- and dual-arm control. On LIBERO, DriftingVLA achieves a 98.32% overall success rate, compared with 97.10% for the original 10-NFE $\pi_{0.5}$ policy and 97.48% for SnapFlow. On RoboTwin 2.0, it reaches 81.09%, exceeding 10-NFE $\pi_{0.5}$ by 1.51 percentage points and SnapFlow by 2.56 points, while a naive one-step evaluation of the conventionally trained $\pi_{0.5}$ flow policy drops to 63.76%. In real-world evaluation, DriftingVLA reaches 77.67% overall success, compared with 74.22% for $\pi_{0.5}$, with improvements on both the single- and dual-arm subsets. The geometry ablation further shows that PDTD consistently improves over both Chunk-wise and Step-wise drifting on LIBERO and RoboTwin 2.0. Finally, under controlled A800 profiling, native one-step generation reduces action-chunk latency from 227.61 ms to 67.67 ms, corresponding to a $3.36\times$ speedup; increasing the number of drifting siblings from $G = 2$ to $G = 8$ raises measured training cost by only 3.78% and peak memory by 9.54%. Together, these results show that iterative action refinement can be removed from deployment without sacrificing control performance.

The present article substantially extends the DBP formulation introduced in DBPO [28] to pretrained VLAs. The resulting extensions and contributions are threefold:

- We extend native drift-based generation to pretrained flow-based VLAs by preserving and jointly adapting the pretrained multimodal backbone while relearning the flow-specific action interface as a direct conditional generator, enabling one-step deployment without numerical integration or flow-trajectory distillation.
- We identify action-chunk organization as a key design variable when extending distribution drifting to VLAs and propose Per-Dimension Temporal Drifting (PDTD). In contrast to the Chunk-wise and Step-wise organizations considered in DBP, PDTD preserves complete per-channel temporal trajectories while separating heterogeneous action-channel geometries, without factorizing the

joint action generator.

- We provide an expanded evaluation across LIBERO, RoboTwin 2.0, and six real-world single- and dual-arm tasks, together with controlled studies of VLA adaptation, drifting geometry, and deployment efficiency. DriftingVLA outperforms representative one-step VLA approaches and multi-step $\pi_{0.5}$ while substantially reducing action-generation latency.

II. RELATED WORK

A. Vision-Language-Action Models

The development of vision-language-action (VLA) models builds on advances in large-scale language-conditioned robot learning, cross-embodiment data, and pretrained multimodal representations [1], [2], [4]. RT-2 [3] demonstrated how vision-language models can be co-fine-tuned on web-scale data and robotic trajectories for robotic control, while Octo [5] and OpenVLA [6] further developed large-scale, reusable robot policies across diverse tasks and embodiments. A central design choice in these models is the action interface [7]–[9], [11], [34]. FAST [32] compresses action sequences into frequency-space tokens for efficient autoregressive control, whereas π_0 [12] and $\pi_{0.5}$ [13] pair a pretrained multimodal backbone with a flow-matching action expert for continuous action-chunk generation. These developments highlight a useful separation between multimodal representation learning and action generation. DriftingVLA focuses on the latter, retaining the pretrained multimodal representation while redesigning the continuous action generator for direct one-step control.

B. Efficient and One-Step Generative Action Policies

The inference cost of iterative generative policies has motivated several forms of acceleration [16], [17], [20], [26], [35], [36]. For diffusion-based visuomotor policies, Consistency Policy [14] and One-Step Diffusion Policy [15] distill pretrained diffusion policies into few- or one-step generators. For flow-based VLAs, execution-level approaches instead retain the iterative generator while reducing its impact on online control. Real-Time Chunking (RTC) [18] overlaps action generation with execution, while FASTER [19] prioritizes near-term actions through horizon-aware sampling. These approaches improve responsiveness without replacing the underlying iterative generative process.

Other methods directly target one-step action generation through changes to the training or action-generation formulation [17], [21]–[23], [37]–[39]. SnapFlow [24] uses progressive self-distillation to compress multi-step flow sampling into 1-NFE generation, while MeanFlowVLA [25] adapts the MeanFlow principle [26] to predict interval-averaged velocity for one-step action generation. π_0 -EqM [40] explores an alternative equilibrium-matching decoder with adaptive inference depth. DriftingVLA differs in how one-step behavior is obtained: instead of deriving the deployed generator from an inference-time transport process, it trains a direct conditional generator whose output distribution is refined during optimization.

C. Drifting Models and One-Step Robot Policies

Drifting Models [27] formulate generative learning through the pushforward distribution of a generator, using distribution-level corrections during training instead of iterative denoising or transport at inference. Drift-Based Policy Optimization (DBPO) [28] introduced a native one-step policy learning framework for robotic control, whose generative component, Drift-Based Policy (DBP), applies distribution drifting in action space using multiple sibling action hypotheses during training while retaining single-sample, one-evaluation deployment.

DriftingVLA builds on the DBP generative formulation and studies the additional issues that arise when distribution drifting is extended to a pretrained flow-based VLA. First, the flow-specific action interface must be converted into a direct generator without discarding the pretrained multimodal representation. Second, the structured $H \times D$ action chunk requires an explicit choice of distributional geometry. Whereas DBP considers Chunk-wise and Step-wise organizations for action-chunk drifting, DriftingVLA introduces Per-Dimension Temporal Drifting (PDTD), which defines channel-specific geometry over complete temporal trajectories while retaining joint action generation through the shared VLA model.

III. PRELIMINARIES

We first review the flow-based action generation mechanism used by $\pi_{0.5}$ and the drift-based policy learning framework introduced in prior work. Throughout the paper, we denote the conditioning context at control step t by $\mathbf{c}_t = (\mathbf{I}_t^{1:N_v}, \ell_t, \mathbf{s}_t)$, which consists of multi-view visual observations $\mathbf{I}_t^{1:N_v}$, a language instruction ℓ_t , and the robot proprioceptive state $\mathbf{s}_t$. The policy predicts an action chunk

$$\mathbf{A}_t = [\mathbf{a}_t^1, \dots, \mathbf{a}_t^H]^\top \in \mathbb{R}^{H \times D}, \quad (1)$$

where $\mathbf{a}_t^h \in \mathbb{R}^D$ denotes the action at future step h , H denotes the prediction horizon, and D denotes the per-step action dimension. We use $h \in \{1, \dots, H\}$ and $d \in \{1, \dots, D\}$ to index the temporal and action-dimension axes, respectively.

A. Flow-Based Action Generation in $\pi_{0.5}$

$\pi_{0.5}$ [13] builds on the flow-based VLA architecture introduced by π_0 [12]. Its low-level continuous action generator consists of a pretrained vision-language backbone together with a smaller action expert specialized for processing robot states and continuous action tokens. During post-training, the multimodal context forms a shared prefix, while the action expert receives noisy action tokens and predicts the corresponding flow-matching vector field.

Specifically, conditional flow matching [41], [42] constructs an interpolation between Gaussian noise and the demonstrated action chunk. Given a demonstration $\mathbf{A}_t$ and Gaussian noise $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, the noisy action at flow time $\tau \in [0, 1]$ is

$$\mathbf{A}_t^\tau = \tau \mathbf{A}_t + (1 - \tau) \epsilon. \quad (2)$$

The action expert parameterizes a conditional velocity field $\mathbf{v}_\theta(\mathbf{A}_t^\tau, \mathbf{c}_t, \tau)$ and is trained to match the target vector field $\mathbf{A}_t - \epsilon$:

$$\mathcal{L}_{\text{FM}} = \mathbb{E}_{\mathbf{A}_t, \mathbf{c}_t, \tau, \epsilon} \left[\|\mathbf{v}_\theta(\mathbf{A}_t^\tau, \mathbf{c}_t, \tau) - (\mathbf{A}_t - \epsilon)\|_F^2 \right]. \quad (3)$$

Here, θ denotes the model parameters and $\|\cdot\|_F$ denotes the Frobenius norm over the action chunk.

At inference time, action generation starts from $\mathbf{A}_t^0 \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and numerically integrates the learned vector field from $\tau = 0$ to $\tau = 1$. Using forward Euler integration with K steps and step size $\delta = 1/K$,

$$\mathbf{A}_t^{\tau_{k+1}} = \mathbf{A}_t^{\tau_k} + \delta \mathbf{v}_\theta(\mathbf{A}_t^{\tau_k}, \mathbf{c}_t, \tau_k), \quad \tau_k = \frac{k}{K}. \quad (4)$$

The final action chunk is obtained as $\hat{\mathbf{A}}_t = \mathbf{A}_t^1$. Following π_0 , the multimodal prefix can be cached across integration steps, such that repeated computation is concentrated in the action expert. Nevertheless, generating one action chunk still requires K successive evaluations of the action expert, motivating action-generation mechanisms that avoid iterative inference altogether.

B. Drift-Based Policy Learning

Drifting Models (DM) [27] provide a different generative principle in which distribution refinement occurs through optimization rather than through an iterative inference trajectory. Drift-Based Policy (DBP), introduced as the generative component of Drift-Based Policy Optimization (DBPO) [28], specializes this principle to robotic control and enables direct one-step action generation.

We first summarize the underlying drifting formulation. Let $\mathbf{z} \sim p_0$ denote a latent sample from a fixed Gaussian prior and let $\mathbf{x} = f_\theta(\mathbf{z}) \in \mathbb{R}^S$ denote a generic generated sample, where S is the dimensionality of the drifting unit. The generator induces the pushforward distribution

$$q_\theta = [f_\theta]_{\#} p_0. \quad (5)$$

Let p_{data} denote the target data distribution. At optimization iteration k , the current parameters induce a distribution q_k . For a fixed latent seed, DM conceptually evolves the generated sample according to

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \mathcal{V}_{p_{\text{data}}, q_k}(\mathbf{x}_k), \quad (6)$$

where $\mathcal{V}_{p,q}(\mathbf{x})$ is a drifting field that specifies the distribution-level correction at $\mathbf{x}$. DM employs an anti-symmetric construction satisfying $\mathcal{V}_{p,q}(\mathbf{x}) = -\mathcal{V}_{q,p}(\mathbf{x})$, and consequently $\mathcal{V}_{p,p}(\mathbf{x}) = \mathbf{0}$ at distributional equilibrium.

Rather than explicitly applying Eq. (6) during inference, the correction is converted into a stop-gradient regression target during training:

$$\tilde{\mathbf{x}} = \text{sg}(f_\theta(\mathbf{z}) + \mathcal{V}_{p_{\text{data}}, q_\theta}(f_\theta(\mathbf{z}))), \quad (7)$$

where $\text{sg}(\cdot)$ denotes the stop-gradient operation. The generator is optimized by

$$\mathcal{L}_{\text{DM}} = \mathbb{E}_{\mathbf{z} \sim p_0} \left[\|\mathbf{f}_\theta(\mathbf{z}) - \tilde{\mathbf{x}}\|_2^2 \right]. \quad (8)$$

Repeated optimization progressively absorbs these distributional corrections into the generator parameters. Consequently, once training is complete, generation requires only the direct mapping $f_\theta(\mathbf{z})$, without performing the drifting updates in Eq. (6) at inference time.

In practice, the drifting field is estimated through interactions between target and generated samples. A generic kernelized form is

$$\mathcal{V}_{p,q}(\mathbf{x}) = \mathbb{E}_{\mathbf{y}^+ \sim p, \mathbf{y}^- \sim q} [\mathcal{K}(\mathbf{x}, \mathbf{y}^+, \mathbf{y}^-)], \quad (9)$$

where $\mathbf{y}^+$ and $\mathbf{y}^-$ denote target and generated references, respectively. The resulting interaction combines attraction toward target samples with repulsion from generated samples while preserving the anti-symmetric drifting construction.

DBP applies this principle conditionally in robot action space. Given a training pair $(\mathbf{c}_i, \mathbf{A}_i)$, it draws G independent latent samples and generates multiple sibling hypotheses,

$$\mathbf{Z}_i^{(g)} \stackrel{\text{i.i.d.}}{\sim} p_0, \quad \hat{\mathbf{A}}_i^{(g)} = f_\theta(\mathbf{Z}_i^{(g)}, \mathbf{c}_i), \quad g = 1, \dots, G, \quad (10)$$

which provide samples from the conditional generated distribution

$$q_\theta(\cdot | \mathbf{c}_i) = [f_\theta(\cdot, \mathbf{c}_i)]_{\#} p_0. \quad (11)$$

The drifting objective then shapes these hypotheses using demonstrated action chunks as positive references and generated hypotheses as negative references. Importantly, the multiple siblings are required only during training. At deployment, a single latent sample $\mathbf{Z}_t$ is drawn and a complete action chunk is obtained through one evaluation,

$$\hat{\mathbf{A}}_t = f_\theta(\mathbf{Z}_t, \mathbf{c}_t). \quad (12)$$

DBP considers two ways of organizing an action chunk for drifting. In Chunk-wise Drifting, the complete action chunk is flattened and treated as a single drifting unit,

$$\mathbf{x}_{\text{chunk}} = \text{vec}(\mathbf{A}_t) \in \mathbb{R}^{HD}. \quad (13)$$

In Step-wise Drifting, the drifting objective is instead applied independently to each temporal slice,

$$\mathbf{x}_{\text{step}}^h = \mathbf{A}_{t,h,:} \in \mathbb{R}^D, \quad h = 1, \dots, H, \quad (14)$$

and the resulting losses are averaged across the prediction horizon. These two organizations define the action-chunk groupings considered in DBP. In the next section, we revisit this design when extending DBP to pretrained VLA models.

IV. METHOD

We introduce DriftingVLA, a VLA policy with native one-step action generation. Extending the DBP formulation in Sec. III-B to a pretrained flow-based VLA requires addressing two coupled design problems: adapting the flow-specific action interface to direct generation while retaining pretrained multimodal representations, and defining an appropriate drifting geometry for the structured action chunk. DriftingVLA addresses the first by reconfiguring the $\pi_{0.5}$ action-generation branch as a direct noise-to-action generator and jointly post-training it with the pretrained VLM, and the second through Per-Dimension Temporal Drifting (PDTD), which organizes distribution drifting along each action channel's complete temporal trajectory.

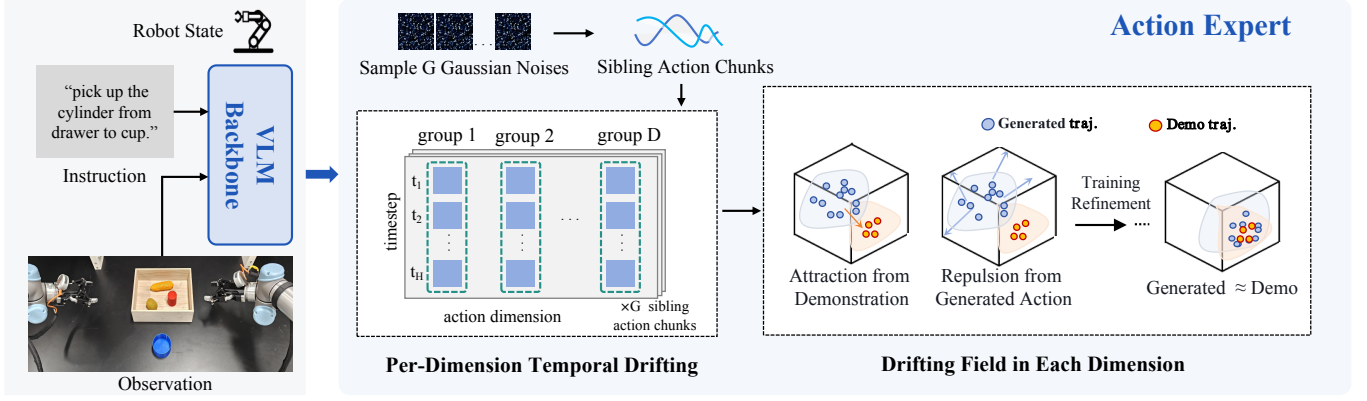


Fig. 2. Training pipeline of DriftingVLA. The pretrained $\pi_{0.5}$ multimodal backbone encodes each condition once, and the resulting shared prefix is reused across G sibling generations from independent Gaussian latent samples. A freshly initialized one-step action expert generates sibling action chunks in parallel, which are reorganized by PDTD into per-dimension temporal trajectories. The drifting field constructs attraction–repulsion targets from generated siblings and demonstrations, and the resulting detached targets jointly post-train the VLM backbone and action expert.

A. From Flow-Based VLA Models to Direct Action Generation in DriftingVLA

a) *Direct conditional generation*: At control step t , DriftingVLA generates a complete action chunk $\hat{\mathbf{A}}_t \in \mathbb{R}^{H \times D}$ from the multimodal context $\mathbf{c}_t$. Instead of learning the time-dependent velocity field used by $\pi_{0.5}$ and recovering the final action through numerical integration, we directly parameterize the deployed policy as

$$\hat{\mathbf{A}}_t = f_{\theta}(\mathbf{Z}_t, \mathbf{c}_t), \quad \mathbf{Z}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \quad (15)$$

where $\mathbf{Z}_t \in \mathbb{R}^{H \times D}$ is an action-shaped latent sampled from the Gaussian prior p_0 introduced in Sec. III-B. The output of f_{θ} is interpreted directly as the final action chunk rather than as a local update along a generative trajectory. Hence, the mapping learned during training is also the mapping executed at deployment.

This distinction separates DriftingVLA from evaluating a conventionally trained flow policy with only one Euler step. The latter changes the numerical solver while retaining a velocity estimator trained for local transport. DriftingVLA changes the learned function itself: the action expert is optimized from the outset to map latent noise directly to an action chunk. Its one-step behavior is therefore a property of the training formulation, not an inference-time approximation.

b) *Preserving the pretrained VLA representation*: We retain the two-component $\pi_{0.5}$ architecture and write

$$\theta = (\theta_{\text{VLM}}, \theta_{\text{AE}}), \quad (16)$$

where θ_{VLM} denotes the pretrained multimodal backbone and θ_{AE} denotes the continuous action-generation branch. We initialize θ_{VLM} from pretrained $\pi_{0.5}$, while freshly initializing the action expert together with its continuous-action input/output projections. During post-training, both parameter sets are optimized jointly.

This design preserves the pretrained multimodal representation while relearning the generative action interface. The pretrained VLM already provides vision-language representations suitable for conditioning robot behavior, whereas the original action expert is specialized for estimating a flow vector field.

Retaining the former while relearning the latter avoids discarding the pretrained semantic and perceptual priors, yet does not constrain the new generator to inherit the previous velocity-field parameterization.

c) *Shared-context sibling generation*: Distribution drifting requires multiple samples from the current conditional generator during training. For each training pair $(\mathbf{c}_i, \mathbf{A}_i)$, we draw G independent latent chunks as in Eq. (10). Since all siblings share the same multimodal condition, we factor generation into a shared context encoder and sibling-specific action generation:

$$\begin{aligned} \mathbf{P}_i &= E_{\theta_{\text{VLM}}}(\mathbf{c}_i), \\ \hat{\mathbf{A}}_i^{(g)} &= F_{\theta_{\text{AE}}}(\mathbf{Z}_i^{(g)}; \mathbf{P}_i), \quad g = 1, \dots, G. \end{aligned} \quad (17)$$

Here, $E_{\theta_{\text{VLM}}}$ denotes the multimodal context encoder and $F_{\theta_{\text{AE}}}$ denotes the action-generation branch conditioned on the shared context. The context is reused across all G siblings, avoiding repeated multimodal computation, while gradients from all sibling losses jointly update the pretrained VLM during post-training. Because the siblings differ only in their independently sampled action latents, they remain samples from the same conditional generator $q_{\theta}(\cdot | \mathbf{c}_i)$ defined in Sec. III-B. Thus, shared-context sampling reduces the cost of conditional distribution learning without changing the conditional distribution being optimized.

Figure 2 summarizes the complete DriftingVLA training pipeline.

The formulation above specifies how drifting is integrated into a pretrained VLA, while the organization of the $H \times D$ action chunk remains an important design choice in the drifting objective. This grouping determines which coordinates share pairwise distances, scale statistics, affinities, and normalized drifting forces.

B. From Action-Chunk Geometry to Per-Dimension Temporal Drifting

An action chunk $\mathbf{A} \in \mathbb{R}^{H \times D}$ has two structurally different axes [7], [29]–[33]. The temporal slice $\mathbf{A}_{h,:} \in \mathbb{R}^D$ collects heterogeneous control variables at one future step, whereas the

column $\mathbf{A}_{:,d} \in \mathbb{R}^H$ describes the complete temporal evolution of one action channel. This distinction matters for drifting because distances determine the neighborhood structure used to construct attraction and repulsion updates.

The two organizations reviewed in Sec. III-B induce different geometries. Chunk-wise drifting uses the entire flattened chunk as one HD -dimensional unit. For two chunks $\mathbf{A}$ and $\mathbf{B}$,

$$\|\text{vec}(\mathbf{A}) - \text{vec}(\mathbf{B})\|_2^2 = \sum_{d=1}^D \sum_{h=1}^H (A_{h,d} - B_{h,d})^2. \quad (18)$$

It therefore preserves the complete horizon, but all action channels jointly determine a single distance, a single scale, and the same distribution-level neighborhood statistics. A discrepancy in one channel changes the geometry that governs the drifting signal applied to every other channel. We refer to this effect as cross-channel geometric interference. This does not imply that cross-channel coupling in the policy is undesirable; coordinated robot control clearly requires it. The issue is specifically whether heterogeneous control channels must also share one distributional metric and normalization.

Step-wise drifting reduces the scope of each group by treating $\mathbf{A}_{h,:}$ independently for $h = 1, \dots, H$. This allows different future timesteps to have separate drifting statistics, but fragments the complete trajectory $\mathbf{A}_{:,d} = [A_{1,d}, \dots, A_{H,d}]^\top$ across H independently normalized problems. Consequently, the drifting objective no longer directly compares two samples according to the full temporal evolution of channel d . Moreover, all D heterogeneous channels at a given timestep still jointly determine the same local distance.

These limitations motivate organizing the action tensor along its remaining natural orientation. We introduce Per-Dimension Temporal Drifting (PDTD), which treats the full temporal trajectory of each action dimension as one drifting unit:

$$\mathcal{G}_{\text{PDTD}}(\mathbf{A}) = \{\mathbf{A}_{:,d}\}_{d=1}^D, \quad \mathbf{A}_{:,d} \in \mathbb{R}^H. \quad (19)$$

Hence, Chunk-wise, Step-wise, and PDTD respectively define drifting units of dimensions HD , D , and H . More importantly, they assign the temporal and action-channel axes different roles: Chunk-wise collapses both axes into one geometry; Step-wise indexes groups by time and mixes channels inside each group; PDTD indexes groups by action channel and preserves the complete temporal horizon inside each group.

Figure 3 visualizes the three organizations and their corresponding group shapes.

For sibling g of training example i , PDTD extracts

$$\mathbf{x}_{i,g}^{(d)} = \hat{\mathbf{A}}_{i,+,d}^{(g)}, \quad \mathbf{x}_{i,+}^{(d)} = \mathbf{A}_{i,+,d}, \quad (20)$$

so that, for every channel d , the G siblings provide multiple predicted H -step trajectories under the same condition. Pairwise distances, data-dependent distance scales, affinities, and force normalizations are then estimated independently for each d . PDTD therefore gives each action channel its own temporal drifting geometry while retaining the complete trajectory as the primitive distributional object.

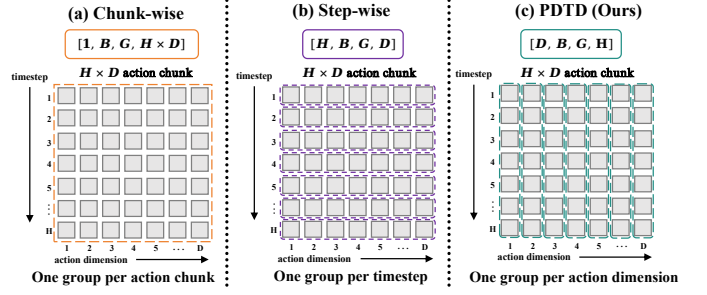


Fig. 3. Action-chunk grouping strategies for distribution drifting. Chunk-wise drifting treats an $H \times D$ action chunk as one HD -dimensional group; Step-wise drifting forms H groups of dimension D ; and PDTD forms D groups of dimension H . PDTD preserves the complete temporal trajectory of each action channel while defining its drifting geometry separately. The grouping affects only the drifting geometry; the full action chunk is still generated jointly by the shared VLA model.

This organization is particularly suitable for VLA action interfaces whose channels correspond to heterogeneous controls such as translation, rotation, gripper commands, or other embodiment-dependent quantities. These channels can differ substantially in numerical range and in the temporal variations that characterize similar behavior. PDTD therefore allows each channel to maintain its own trajectory scale and similarity structure while still using the complete temporal trajectory as the primitive distributional object. The grouping imposes no particular temporal profile; it only determines how distributional similarity is measured when constructing the drifting signal.

a) Factorized geometry, joint action generation: A crucial distinction is that PDTD factorizes the drifting geometry, not the action generator. The complete action chunk is generated jointly before any grouping is applied:

$$\hat{\mathbf{A}}_i^{(g)} = f_{\theta}(\mathbf{Z}_i^{(g)}, \mathbf{c}_i) \in \mathbb{R}^{H \times D}. \quad (21)$$

PDTD is applied only afterward to construct the training signal. Accordingly, we neither assume nor enforce

$$q_{\theta}(\mathbf{A} | \mathbf{c}) = \prod_{d=1}^D q_{\theta}(\mathbf{A}_{:,d} | \mathbf{c}). \quad (22)$$

All per-dimension drifting losses update the same joint VLA generator. Thus, the prediction of one channel is not restricted to depend only on the corresponding latent coordinates, and cross-dimensional dependencies can still be represented through the shared conditional generator. This distinction is important for coordinated multi-axis and multi-joint control, where different control channels must remain coupled at the policy level even though their drifting geometries are defined separately. In short, PDTD removes geometry-level coupling without removing model-level coupling.

C. Per-Dimension Drifting Objective

We next specify how the drifting field is constructed once PDTD has defined the groups. The formulation follows the attraction and repulsion mechanism reviewed in Sec. III-B; here we emphasize the quantities that become channel specific under PDTD. For a minibatch of B conditions, all trajectory

statistics and reductions below are evaluated only over valid temporal coordinates, with padded entries excluded.

For each condition $\mathbf{c}_i$ and channel d , we form a reference set containing the G generated sibling trajectories and the demonstrated trajectory,

$$\mathcal{Y}_i^{(d)} = \{\mathbf{x}_{i,1}^{(d)}, \dots, \mathbf{x}_{i,G}^{(d)}, \mathbf{x}_{i,+}^{(d)}\}. \quad (23)$$

Let $\mathbf{y}_{i,u}^{(d)}$ denote its u -th element. PDTD first estimates a separate distance ruler for every action channel,

$$s_d = \text{Mean}_{i,g,u} \left[\left\| \mathbf{x}_{i,g}^{(d)} - \mathbf{y}_{i,u}^{(d)} \right\|_2 \right], \quad (24)$$

$$\Delta_{i,g,u}^{(d)} = \frac{\left\| \mathbf{x}_{i,g}^{(d)} - \mathbf{y}_{i,u}^{(d)} \right\|_2}{\max(s_d, \varepsilon)}.$$

Here and below, small positive constants are used where needed for numerical stability. The ruler s_d is estimated from the current generated–reference relations for channel d , so the normalized distance reflects trajectory variation within that channel rather than the aggregate scale of the full action tensor. A channel with a different numerical range or trajectory variability therefore obtains a different geometric ruler rather than inheriting the scale of other action dimensions.

From the normalized distances, we construct multi-scale attraction and repulsion interactions for bandwidths $\rho \in \mathcal{R}$. Let $\mathcal{I}^- = \{1, \dots, G\}$ denote generated references and $\mathcal{I}^+ = \{G+1\}$ the demonstrated reference. Self-connections between a sibling and its identical generated reference are excluded by defining

$$\tilde{\Delta}_{i,g,u}^{(d)} = \begin{cases} +\infty, & u = g, u \in \mathcal{I}^-, \\ \Delta_{i,g,u}^{(d)}, & \text{otherwise,} \end{cases} \quad \ell_{i,g,u}^{(\rho,d)} = -\frac{\tilde{\Delta}_{i,g,u}^{(d)}}{\rho}. \quad (25)$$

For each generated trajectory, one Softmax normalizes over candidate references and the other over competing generated siblings for a fixed reference:

$$P_{i,g,u}^{\rightarrow(\rho,d)} = \frac{\exp(\ell_{i,g,u}^{(\rho,d)})}{\sum_{u'} \exp(\ell_{i,g,u'}^{(\rho,d)})},$$

$$P_{i,g,u}^{\leftarrow(\rho,d)} = \frac{\exp(\ell_{i,g,u}^{(\rho,d)})}{\sum_{g'} \exp(\ell_{i,g',u}^{(\rho,d)})}, \quad (26)$$

$$W_{i,g,u}^{(\rho,d)} = \sqrt{\max(P_{i,g,u}^{\rightarrow(\rho,d)} P_{i,g,u}^{\leftarrow(\rho,d)}, \varepsilon_a)}.$$

The geometric mean yields a bidirectionally normalized affinity, assigning large weight only when the generated trajectory and the reference mutually favor the interaction. Applying this construction independently for every d gives each action channel its own neighborhood structure, while using multiple bandwidths allows the field to aggregate interactions at different trajectory scales.

We next balance attraction to demonstrations and repulsion among generated siblings. Define the negative and positive affinity masses

$$S_{i,g,-}^{(\rho,d)} = \sum_{u \in \mathcal{I}^-} W_{i,g,u}^{(\rho,d)}, \quad S_{i,g,+}^{(\rho,d)} = \sum_{u \in \mathcal{I}^+} W_{i,g,u}^{(\rho,d)}. \quad (27)$$

The signed coefficients are

$$\alpha_{i,g,u}^{(\rho,d)} = \begin{cases} -W_{i,g,u}^{(\rho,d)} S_{i,g,+}^{(\rho,d)}, & u \in \mathcal{I}^-, \\ W_{i,g,u}^{(\rho,d)} S_{i,g,-}^{(\rho,d)}, & u \in \mathcal{I}^+. \end{cases} \quad (28)$$

Thus generated references contribute repulsive coefficients whereas the demonstrated reference contributes an attractive coefficient. The cross-side mass coupling gives

$$\sum_{u \in \mathcal{I}^-} \alpha_{i,g,u}^{(\rho,d)} = - \sum_{u \in \mathcal{I}^+} \alpha_{i,g,u}^{(\rho,d)}, \quad \sum_u \alpha_{i,g,u}^{(\rho,d)} = 0, \quad (29)$$

so the field is translation invariant and can be expressed through relative displacements. This balance also prevents the net coefficient mass from being dominated by one side of the attraction–repulsion interaction.

For force construction, we normalize trajectory coordinates using

$$\eta_d = \max(s_d / \sqrt{H}, \varepsilon), \quad \bar{\mathbf{x}}_{i,g}^{(d)} = \frac{\mathbf{x}_{i,g}^{(d)}}{\eta_d}, \quad \bar{\mathbf{y}}_{i,u}^{(d)} = \frac{\mathbf{y}_{i,u}^{(d)}}{\eta_d}. \quad (30)$$

The scale-specific force and the resulting multi-scale drifting field are

$$\mathbf{F}_{i,g}^{(\rho,d)} = \sum_{u=1}^{G+1} \alpha_{i,g,u}^{(\rho,d)} (\bar{\mathbf{y}}_{i,u}^{(d)} - \bar{\mathbf{x}}_{i,g}^{(d)}),$$

$$r_{\rho,d} = \text{RMS}_{i,g} (\mathbf{F}_{i,g}^{(\rho,d)}), \quad (31)$$

$$\mathbf{V}_{i,g}^{(d)} = \sum_{\rho \in \mathcal{R}} \frac{\mathbf{F}_{i,g}^{(\rho,d)}}{\max(r_{\rho,d}, \varepsilon_f)}.$$

Here, the RMS is evaluated over the minibatch, sibling samples, and valid temporal coordinates. The multi-bandwidth construction combines local and broader neighborhood information, while the per- (ρ, d) normalization prevents a particular scale or action channel from dominating because of its raw numerical magnitude. Together with the channel-specific ruler s_d , this normalization carries the per-dimension factorization through the complete drifting-field construction rather than applying it only at the initial grouping step. Thus, PDTD makes the complete sequence of geometric statistics—distance scale, affinity, attraction–repulsion coupling, and force normalization—channel specific.

As in DBP, the field is converted into a frozen regression target rather than differentiated through:

$$\tilde{\mathbf{x}}_{i,g}^{(d)} = \text{sg} [\bar{\mathbf{x}}_{i,g}^{(d)} + \mathbf{V}_{i,g}^{(d)}]. \quad (32)$$

The PDTD objective is the mean squared regression error over all valid trajectory coordinates,

$$\mathcal{L}_{\text{PDTD}} = \text{Mean}_{i,d,g,h \in \text{valid}(i)} \left[\left(\bar{\mathbf{x}}_{i,g,h}^{(d)} - \tilde{\mathbf{x}}_{i,g,h}^{(d)} \right)^2 \right]. \quad (33)$$

The stop-gradient target is recomputed from samples of the current generator at every optimization iteration. Each update therefore regresses the current sibling trajectories toward targets constructed from the current conditional distribution and the demonstrated trajectory. Repeated optimization absorbs these distribution-level corrections into the generator parameters, following the DBP principle reviewed in Sec. III-B.

Distribution refinement consequently occurs across parameter updates during training; the drifting field itself is not evaluated when the trained policy is deployed.

D. Training and Native One-Step Inference

a) Padding-aware action space: PDTD is evaluated only over valid physical action dimensions and valid temporal coordinates. When the underlying VLA uses a padded action representation to accommodate different embodiments or variable valid horizons, generated and demonstrated chunks are restricted to their valid coordinates before the PDTD geometry is constructed. Padded action dimensions and future timesteps therefore contribute neither to distance statistics nor to affinities or drifting forces.

b) Post-training: During post-training, each condition produces G sibling action chunks from independent latent samples. PDTD reorganizes these chunks into per-dimension temporal trajectories, constructs channel-specific drifting targets from the siblings and the demonstrated chunk, and applies Eq. (33) to update the policy. The resulting objective jointly adapts the pretrained VLM and the freshly initialized action-generation branch. The multimodal context is shared across siblings, so training remains multi-sample and distribution aware without repeating the condition encoding for each hypothesis.

c) Inference: At deployment, all distribution-level machinery used above is removed. Given the current context $\mathbf{c}_t$, we draw one Gaussian latent chunk and evaluate Eq. (15) once. No sibling set, pairwise interaction, drifting field, or numerical integration is required.

Consequently, DriftingVLA moves iterative refinement from robot-time inference to optimization-time distribution learning. The trained policy preserves the multimodal representation and joint action-generation capacity of the pretrained VLA, while deployment reduces to a single direct noise-to-action evaluation.

V. EXPERIMENTS

We evaluate DriftingVLA from five complementary perspectives: (1) whether native one-step action generation can match or exceed iterative flow-based VLA control in simulation, (2) whether this performance transfers to real-world manipulation, (3) whether the proposed PDTD geometry improves over the Chunk-wise and Step-wise organizations inherited from DBP, (4) which components are important for adapting a pretrained VLA to the direct drifting objective, and (5) whether native one-step generation provides the intended deployment-time benefit without excessive training overhead.

A. Experimental Setup

a) Simulation benchmarks: Our simulation evaluation uses LIBERO [43] and RoboTwin 2.0 [44]. LIBERO contains four suites—Spatial, Object, Goal, and Long—with 10 tasks per suite. Each trained policy is evaluated for 100 episodes per task. RoboTwin 2.0 provides a substantially larger and more diverse bimanual manipulation benchmark. We evaluate

all 50 tasks under both the Easy setting (`demo_clean`) and the domain-randomized Hard setting (`demo_randomized`), with 100 evaluation episodes for every task and difficulty level.

For both simulation benchmarks, every model is trained with three random seeds (42, 43, and 44) for 50k optimization steps. Unless otherwise stated, training uses 8 NVIDIA A800 80GB GPUs with a per-GPU batch size of 4. DriftingVLA uses $G = 8$ generated siblings per condition. We report the mean success rate and sample standard deviation across the three seeds.

b) Real-world tasks: We further evaluate on six real-world manipulation tasks, comprising two single-arm tasks and four dual-arm tasks. The platform consists of two UR5 manipulators, two wrist-mounted Intel RealSense L515 cameras, and one head-mounted Orbbec Gemini camera. Single-arm tasks use only the right arm, whereas dual-arm tasks require coordinated control of both manipulators. Following the task order used throughout the real-world evaluation, T1–T2 are Cylinder to Cup and Bottle in Cup, while T3–T6 are Block Alignment, Tabletop Cleanup, Liquid Pouring, and Block Stacking. Figure 5 shows the initial and successful terminal states for all six tasks. We collect 100 teleoperated demonstrations for each task. Each method is trained for 50k optimization steps with three random seeds and evaluated for 50 trials per task and seed, yielding 300 evaluation trials per seed across the six-task suite. Robot-time inference runs on an NVIDIA RTX 3090.

c) Compared methods: For the main simulation comparison, we use the original multi-step $\pi_{0.5}$ [13] with 10 action-expert network function evaluations (NFEs) per generated chunk, together with $\pi_{0.5}$ -1step, obtained by evaluating the same conventionally trained flow policy with a single Euler step. We include the latter as a diagnostic solver-truncation control rather than as a natively trained one-step baseline. We also compare with two representative one-step VLA approaches: SnapFlow [24], which compresses flow-based generation through progressive self-distillation, and MeanFlowVLA [25], which adapts the MeanFlow formulation to one-step VLA generation. We reimplement both one-step baselines following their published descriptions. For the geometry study, DriftingVLA-Chunk and DriftingVLA-Step replace PDTD with the Chunk-wise and Step-wise group organizations considered in DBP [28], respectively, while keeping the remaining DriftingVLA framework unchanged. All compared VLA policies use the same LeRobot $\pi_{0.5}$ base implementation and are trained under matched data and optimization budgets unless explicitly noted.

B. Simulation Benchmark Results

Table I reports the LIBERO comparison. DriftingVLA achieves an overall success rate of 98.32%, outperforming the 10-NFE $\pi_{0.5}$ policy by 1.22 percentage points and the strongest one-step baseline, SnapFlow, by 0.84 points. The margin is particularly informative on LIBERO-Long, where the benchmark is less saturated: DriftingVLA reaches 94.33%, compared with 92.40% for $\pi_{0.5}$ and 93.07% for SnapFlow. In contrast, the $\pi_{0.5}$ -1step solver-truncation control reaches only 94.02% overall. Thus, the one-step inference budget alone does not account for the performance of the direct generator.

TABLE I

LIBERO SUCCESS RATES (%) ACROSS THE SPATIAL, OBJECT, GOAL, AND LONG SUITES. RESULTS ARE REPORTED AS MEAN $\pm$ SAMPLE STANDARD DEVIATION OVER THREE TRAINING SEEDS, WITH 100 EVALUATION EPISODES PER TASK. THE BEST MEAN RESULT IN EACH COLUMN IS SHOWN IN BOLD.

Method	NFE	Success rate (%)				
		Spatial	Object	Goal	Long	Overall
$\pi_{0.5}$	10	98.40 $\pm$ 0.20	99.47 $\pm$ 0.31	98.13 $\pm$ 0.12	92.40 $\pm$ 0.35	97.10 $\pm$ 0.13
$\pi_{0.5}$ -1step	1	95.27 $\pm$ 0.42	96.00 $\pm$ 0.20	94.33 $\pm$ 0.42	90.47 $\pm$ 0.31	94.02 $\pm$ 0.13
SnapFlow	1	99.20 $\pm$ 0.53	98.73 $\pm$ 0.50	98.93 $\pm$ 0.23	93.07 $\pm$ 0.23	97.48 $\pm$ 0.16
MeanFlowVLA	1	94.87 $\pm$ 0.42	94.33 $\pm$ 0.12	93.20 $\pm$ 0.40	89.00 $\pm$ 0.80	92.85 $\pm$ 0.35
DriftingVLA	1	99.73 $\pm$ 0.12	99.87 $\pm$ 0.12	99.33 $\pm$ 0.23	94.33 $\pm$ 0.42	98.32 $\pm$ 0.13

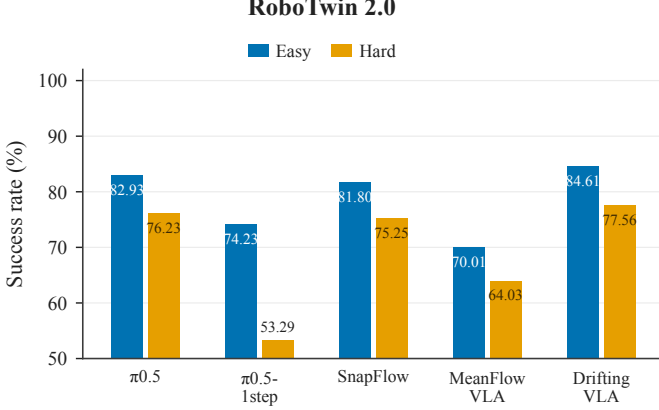


Fig. 4. RoboTwin 2.0 success rates (%) under the Easy and domain-randomized Hard settings. Results are averaged across all 50 tasks and three training seeds. $\pi_{0.5}$ uses 10 NFEs, whereas $\pi_{0.5}$ -1step, SnapFlow, MeanFlowVLA, and DriftingVLA perform one-step action generation.

Figure 4 summarizes the RoboTwin 2.0 comparison, where the separation among methods becomes more pronounced. DriftingVLA reaches 81.09% overall, exceeding 10-NFE $\pi_{0.5}$ by 1.51 points and SnapFlow by 2.56 points. On the harder domain-randomized split, DriftingVLA obtains 77.56%, compared with 76.23% for $\pi_{0.5}$ and 75.25% for SnapFlow. Under the same one-step solver truncation, $\pi_{0.5}$ drops to 63.76% overall, including 53.29% on the Hard split. DriftingVLA also substantially exceeds MeanFlowVLA on both simulation benchmarks. Overall, the main comparison shows that native one-step generation can retain or improve control performance relative to the original iterative policy and representative one-step VLA alternatives, while the large truncation gap motivates the controlled adaptation analysis in Sec. V-D.

C. Real-World Evaluation

Table II reports the real-world results. DriftingVLA reaches the highest overall success rate at 77.67%, improving over 10-NFE $\pi_{0.5}$ (74.22%) by 3.45 percentage points. Relative to the representative external one-step baselines, it improves over SnapFlow and MeanFlowVLA by 8.23 and 12.56 points, respectively. DriftingVLA achieves the best mean success rate on five of the six tasks; on T4 Tabletop Cleanup, $\pi_{0.5}$ is slightly higher (76.67% versus 75.33%).

The physical evaluation also includes the two controlled geometry variants inherited from DBP. DriftingVLA-Chunk reaches 67.78% overall and DriftingVLA-Step reaches

66.11%, so PDTD improves over them by 9.89 and 11.56 points, respectively. The ordering Step < Chunk < PDTD therefore persists from the LIBERO and RoboTwin 2.0 ablations to physical manipulation. This consistency provides additional evidence that action-chunk grouping is a consequential design choice rather than an implementation detail, while the complete simulation ablation in Sec. V-D remains the controlled comparison of the three geometries.

DriftingVLA improves on both the single- and dual-arm subsets. Averaging the two single-arm tasks, success increases from 86.00% for $\pi_{0.5}$ to 89.67% for DriftingVLA, a 3.67-point gain. Across the four dual-arm tasks, the corresponding average increases from 68.34% to 71.67%, a 3.33-point gain. The improvement on the dual-arm subset is relevant because these tasks require coordinated control across both manipulators. Although PDTD factorizes the drifting geometry across action dimensions, the full action chunk is still generated jointly by the shared VLM and Transformer action backbone, as described in Sec. IV-B. The real-world results therefore provide empirical evidence that per-dimension geometric supervision does not prevent coordinated multi-axis action generation.

a) *Failure modes*: Observed failures include incorrect object localization, unstable grasps, slippage, collisions, misalignment, and unstable placement. Dual-arm tasks additionally exhibit arm-assignment, sequencing, inter-arm collision, and coordination failures, while pouring tasks may fail because of inaccurate alignment, insufficient transfer, or spillage. Trials exhibiting these failure modes are counted as unsuccessful.

D. Ablation Studies

We next isolate the two central design decisions of the DriftingVLA framework: the geometry used to organize action chunks for drifting, and the way a pretrained VLA is adapted to the new direct generative objective.

a) *Action-chunk drifting geometry*: Table III, panel (a) compares PDTD with the Chunk-wise and Step-wise geometries considered in DBP under the same DriftingVLA training framework. PDTD improves over Chunk-wise drifting by 3.04 points on LIBERO and 6.92 points on RoboTwin 2.0, and exceeds Step-wise drifting by 5.22 and 10.66 points, respectively. The larger separation on RoboTwin 2.0 shows that the benefit of PDTD remains pronounced on a broader bimanual benchmark with domain randomization. These results support the structural analysis in Sec. IV-B: sharing one geometry across the entire heterogeneous action chunk can introduce cross-channel interference, whereas splitting by timestep fragments

TABLE II

REAL-WORLD SUCCESS RATES (%) ACROSS SIX MANIPULATION TASKS. T1–T2 ARE SINGLE-ARM TASKS AND T3–T6 ARE BIMANUAL TASKS. RESULTS ARE REPORTED AS MEAN $\pm$ SAMPLE STANDARD DEVIATION OVER THREE TRAINING SEEDS, WITH 50 TRIALS PER TASK AND SEED. OVERALL AGGREGATES ALL SIX TASKS (300 TRIALS PER SEED). THE BEST MEAN RESULT IN EACH ROW IS SHOWN IN BOLD.

Task	Reference	One-step baselines		Geometry variants		Ours
	$\pi_{0.5}$	SnapFlow	MeanFlowVLA	DriftingVLA-Chunk	DriftingVLA-Step	DriftingVLA
T1 Cylinder to Cup	84.00 $\pm$ 4.00	74.67 $\pm$ 3.06	76.67 $\pm$ 2.31	76.67 $\pm$ 1.15	77.33 $\pm$ 1.15	87.33 $\pm$ 7.02
T2 Bottle in Cup	88.00 $\pm$ 4.00	83.33 $\pm$ 3.06	82.00 $\pm$ 3.46	86.67 $\pm$ 2.31	84.67 $\pm$ 1.15	92.00 $\pm$ 2.00
T3 Block Alignment	62.67 $\pm$ 5.03	64.00 $\pm$ 5.29	52.67 $\pm$ 8.08	54.67 $\pm$ 6.43	52.67 $\pm$ 8.33	67.33 $\pm$ 10.07
T4 Tabletop Cleanup	76.67 $\pm$ 6.43	68.67 $\pm$ 10.07	66.00 $\pm$ 11.14	67.33 $\pm$ 9.02	66.67 $\pm$ 7.02	75.33 $\pm$ 9.87
T5 Liquid Pouring	66.00 $\pm$ 6.00	59.33 $\pm$ 6.43	52.67 $\pm$ 9.02	58.67 $\pm$ 3.06	57.33 $\pm$ 7.57	72.67 $\pm$ 5.03
T6 Block Stacking	68.00 $\pm$ 9.17	66.67 $\pm$ 7.02	60.67 $\pm$ 9.87	62.67 $\pm$ 8.33	58.00 $\pm$ 9.17	71.33 $\pm$ 2.31
Overall	74.22 $\pm$ 1.17	69.44 $\pm$ 3.10	65.11 $\pm$ 3.75	67.78 $\pm$ 2.04	66.11 $\pm$ 1.71	77.67 $\pm$ 1.76

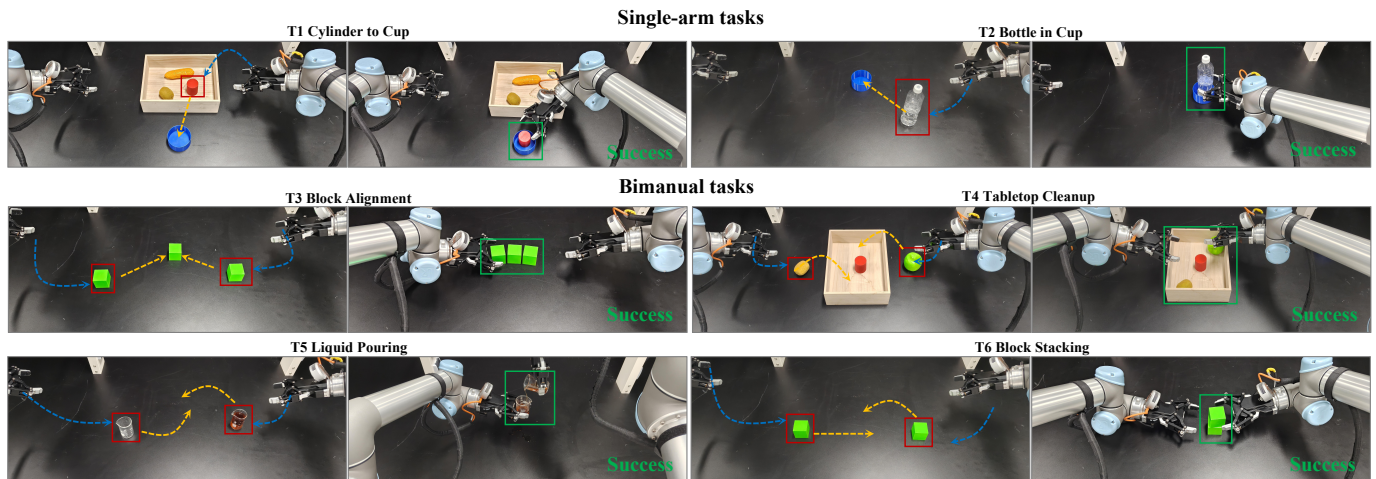


Fig. 5. Real-world evaluation tasks. Initial and successful terminal states are shown for six manipulation tasks: two single-arm tasks (T1–T2) and four dual-arm tasks (T3–T6). The tasks are Cylinder to Cup, Bottle in Cup, Block Alignment, Tabletop Cleanup, Liquid Pouring, and Block Stacking.

the temporal trajectory of each action channel. PDTD retains the full temporal trajectory while allowing channel-specific geometry. The same Step–Chunk–PDTD ordering in the real-world results of Table II further supports the transfer of this geometric advantage beyond simulation.

b) Adapting a pretrained VLA to drifting: Table III, panel (b) evaluates the main choices introduced in Sec. IV-A. First, simply evaluating the trained $\pi_{0.5}$ flow policy with one NFE reduces LIBERO performance from 97.10% to 94.02%, showing that solver truncation alone does not produce a native one-step generator. Second, warm-starting DriftingVLA from the flow-pretrained action expert reaches only 93.33%, compared with 98.32% when the action interface is freshly initialized. This supports relearning the generative action interface rather than inheriting parameters specialized for local flow-velocity estimation. Third, freezing the VLM and training only the fresh action expert yields 94.57%. Jointly post-training the pre-trained VLM and the new action expert improves success by an additional 3.75 points over the Expert-only variant, showing the importance of adapting the multimodal representation to the direct drifting objective.

TABLE III

CONTROLLED ABLATIONS OF DRIFTINGVLA (SUCCESS RATE, %; MEAN $\pm$ SAMPLE STANDARD DEVIATION OVER THREE SEEDS). (A) ACTION-CHUNK DRIFTING GEOMETRY ON LIBERO AND ROBOTWIN 2.0. (B) PRETRAINED VLA ADAPTATION ON LIBERO.

(a) Action-chunk drifting geometry				
Geometry	NFE	LIBERO	RoboTwin 2.0	
Chunk-wise	1	95.28 $\pm$ 0.03	74.17 $\pm$ 0.27	
Step-wise	1	93.10 $\pm$ 0.40	70.43 $\pm$ 0.28	
PDTD (ours)	1	98.32 $\pm$ 0.13	81.09 $\pm$ 0.38	
(b) Pretrained-VLA adaptation on LIBERO				
Variant	Expert init.	VLM updated	NFE	Overall
$\pi_{0.5}$	Pretrained $\pi_{0.5}$	Yes	10	97.10 $\pm$ 0.13
$\pi_{0.5}$ -1step	Pretrained $\pi_{0.5}$	Yes	1	94.02 $\pm$ 0.13
Warm-start	Flow-pretrained	Yes	1	93.33 $\pm$ 0.29
Expert-only	Fresh	No	1	94.57 $\pm$ 0.15
DriftingVLA	Fresh	Yes	1	98.32 $\pm$ 0.13

E. Computational Efficiency

The main motivation for native one-step generation is to remove repeated action-expert evaluations from the online

control loop. We therefore examine DriftingVLA from both deployment-time efficiency and training-time overhead.

a) *Inference latency*: We profile the seed-42, 50k-step checkpoints on LIBERO-Spatial task 0 with batch size 1 and $H = 50$ on a single NVIDIA A800 GPU. Each method is measured over 10 episodes and three independent profiling repetitions, with three warm-up chunk generations excluded from each run. CUDA events are used for model-stage timing and synchronized wall-clock measurements for the surrounding pipeline.

As shown in Fig. 6(a), DriftingVLA reduces mean action-chunk latency from 227.61 ms to 67.67 ms, corresponding to a $3.36\times$ speedup and a 70.3% reduction in latency. This reduction follows directly from replacing the 10-NFE iterative inference of $\pi_{0.5}$ with a single generator evaluation, removing repeated action-expert evaluations from the deployment-time control loop.

b) *Training cost*: Native one-step inference shifts distribution refinement to training, where DriftingVLA uses multiple sibling samples to estimate the drifting field. We therefore examine how this additional sampling affects optimization cost. Using LIBERO with seed 43, $H = 50$, 50k steps, 8 NVIDIA A800 GPUs, and a global batch size of 32, we profile DriftingVLA with $G \in \{2, 4, 8\}$ under an otherwise fixed training configuration; checkpoint saving and evaluation are disabled so that GPU-hours reflect the training loop itself.

Figures 6(b,c) show that increasing the number of siblings introduces only moderate training overhead. Increasing G from 2 to 8 raises the measured training cost from 87.92 to 91.24 GPU-hours, an increase of 3.78%, while peak memory grows from 45.99 to 50.38 GiB, corresponding to 9.54%. At the default $G = 8$, DriftingVLA requires 91.24 GPU-hours under the controlled profiling setup, remaining below the corresponding $\pi_{0.5}$ reference of 108.49 GPU-hours. The mild scaling with G reflects the shared-context sampling design in Sec. IV-A: the expensive VLM context is computed once per condition, while additional siblings primarily expand action-side computation.

c) *Summary*: Overall, the experiments support the central design choices of DriftingVLA: native one-step generation preserves or improves control performance, PDTT outperforms the DBP geometries, and the resulting policy substantially reduces deployment-time latency with moderate training overhead.

VI. CONCLUSION AND LIMITATIONS

This work introduced DriftingVLA, a native one-step VLA that replaces inference-time iterative flow transport with training-time distribution drifting. Starting from pretrained $\pi_{0.5}$ multimodal representations, DriftingVLA relearns the continuous action interface as a direct conditional generator and introduces Per-Dimension Temporal Drifting (PDTT), which defines channel-specific drifting geometry over complete temporal trajectories while preserving joint action generation. Across LIBERO, RoboTwin 2.0, and six real-world single- and dual-arm manipulation tasks, DriftingVLA outperforms the original 10-NFE $\pi_{0.5}$ policy and representative

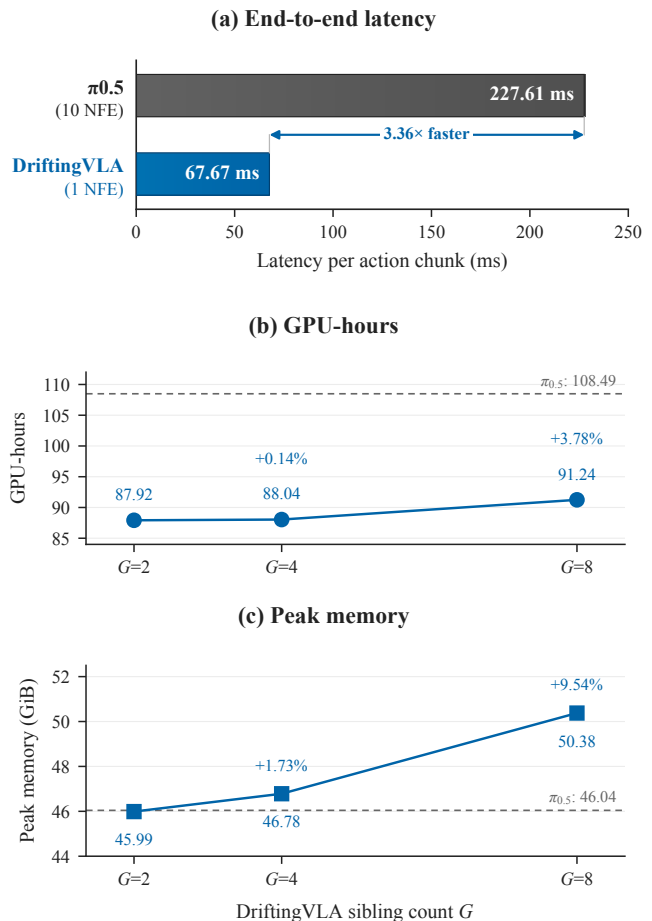


Fig. 6. Computational efficiency of DriftingVLA. (a) End-to-end action-chunk inference latency for 10-NFE $\pi_{0.5}$ and 1-NFE DriftingVLA. (b,c) Training GPU-hours and peak memory of DriftingVLA as the number of drifting siblings increases from $G = 2$ to $G = 8$ under a fixed profiling setup. Dashed lines indicate the corresponding $\pi_{0.5}$ references, and percentage annotations report changes relative to $G = 2$.

one-step baselines in overall success while reducing action-chunk latency from 227.61 ms to 67.67 ms, corresponding to a $3.36\times$ speedup. The ablations further show the importance of PDTT, fresh action-interface learning, and joint VLM adaptation. Together, these results show that native one-step VLA generation can remove iterative action refinement from deployment without sacrificing control performance.

DriftingVLA uses multiple sibling samples during training, introducing additional computation and memory, although this overhead is absent at deployment. Our evaluation also focuses on manipulation tasks and adopts a fixed per-dimension temporal grouping for the action space. Future work can explore adaptive or learned drifting geometries, extend native one-step generation to larger multi-embodiment VLAs with heterogeneous action spaces, and evaluate the approach in broader long-horizon and interactive robotic settings.

REFERENCES

- [1] D. Driess, F. Xia, M. S. M. Sajjadi, C. Lynch, A. Chowdhery, B. Ichter, A. Wahid, J. Tompson, Q. Vuong, T. Yu, W. Huang, Y. Chebotar, P. Sermanet, D. Duckworth, S. Levine, V. Vanhoucke, K. Hausman,

- M. Toussaint, K. Greff, A. Zeng, I. Mordatch, and P. Florence, “Palme: An embodied multimodal language model,” in *Proceedings of the 40th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 202, 2023, pp. 8469–8488.
- [2] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn *et al.*, “Rt-1: Robotics transformer for real-world control at scale,” in *Robotics: Science and Systems*, 2023.
- [3] B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, F. Xia *et al.*, “Rt-2: Vision-language-action models transfer web knowledge to robotic control,” in *Proceedings of The 7th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 229, 2023, pp. 2165–2183.
- [4] Open X-Embodiment Collaboration, A. O’Neill, A. Rehman, A. Gupta, A. Maddukuri *et al.*, “Open x-embodiment: Robotic learning datasets and rt-x models,” in *IEEE International Conference on Robotics and Automation*, 2024.
- [5] D. Ghosh, H. R. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari *et al.*, “Octo: An open-source generalist robot policy,” in *Proceedings of Robotics: Science and Systems*, 2024.
- [6] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair *et al.*, “Openvla: An open-source vision-language-action model,” in *Proceedings of The 8th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 270, 2025, pp. 2679–2713.
- [7] M. J. Kim, C. Finn, and P. Liang, “Fine-tuning vision-language-action models: Optimizing speed and success,” *arXiv preprint arXiv:2502.19645*, 2025.
- [8] S. Liu, L. Wu, B. Li, H. Tan, H. Chen, Z. Wang, K. Xu, H. Su, and J. Zhu, “Rdt-1b: A diffusion foundation model for bimanual manipulation,” *arXiv preprint arXiv:2410.07864*, 2024.
- [9] Q. Li, Y. Liang, Z. Wang, L. Luo, X. Chen, M. Liao, F. Wei, Y. Deng, S. Xu, Y. Zhang, X. Wang, B. Liu, J. Fu, J. Bao, D. Chen, Y. Shi, J. Yang, and B. Guo, “Cogact: A foundational vision-language-action model for synergizing cognition and action in robotic manipulation,” *arXiv preprint arXiv:2411.19650*, 2024.
- [10] M. Shukor, D. Aubakirova, F. Capuano, P. Kooijmans, S. Palma, A. Zoutine, M. Aractingi, C. Pascal, M. Russi, A. Marafioti, S. Alibert, M. Cord, T. Wolf, and R. Cadene, “Smolvla: A vision-language-action model for affordable and efficient robotics,” *arXiv preprint arXiv:2506.01844*, 2025.
- [11] NVIDIA, J. Bjorck, F. Castañeda, N. Cherniadev, X. Da, R. Ding, L. Fan, Y. Fang, D. Fox, F. Hu, S. Huang, J. Jang, Z. Jiang, J. Kautz, K. Kundalia, L. Lao, Z. Li, Z. Lin, K. Lin, G. Liu, E. Llongtop, L. Magne, A. Mandlekar, A. Narayan, S. Nasiriany, S. Reed, Y. L. Tan, G. Wang, Z. Wang, J. Wang, Q. Wang, J. Xiang, Y. Xie, Y. Xu, Z. Xu, S. Ye, Z. Yu, A. Zhang, H. Zhang, Y. Zhao, R. Zheng, and Y. Zhu, “Gr00t n1: An open foundation model for generalist humanoid robots,” *arXiv preprint arXiv:2503.14734*, 2025.
- [12] K. Black, N. Brown, D. Driess, A. Esmail, M. R. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, L. X. Shi, L. Smith, J. Tanner, Q. Vuong, A. Walling, H. Wang, and U. Zhilinsky, “ π_0 : A Vision-Language-Action Flow Model for General Robot Control,” in *Proceedings of Robotics: Science and Systems*, 2025.
- [13] Physical Intelligence, K. Black, N. Brown, J. Darpanian, K. Dhabalia, D. Driess *et al.*, “ $\pi_{0.5}$: A vision-language-action model with open-world generalization,” *arXiv preprint arXiv:2504.16054*, 2025.
- [14] A. Prasad, K. Lin, J. Wu, L. Zhou, and J. Bohg, “Consistency policy: Accelerated visuomotor policies via consistency distillation,” in *Robotics: Science and Systems*, 2024.
- [15] Z. Wang, M. Li, A. Mandlekar, Z. Xu, J. Fan, Y. Narang, L. Fan, Y. Zhu, Y. Balaji, M. Zhou, M.-Y. Liu, and Y. Zeng, “One-step diffusion policy: Fast visuomotor policies via diffusion distillation,” in *International Conference on Machine Learning*, 2025.
- [16] H. Chen, M. Liu, C. Ma, X. Ma, Z. Ma, H. Wu, Y. Chen, Y. Zhong, M. Wang, Q. Li, and Y. Yang, “Falcon: Fast visuomotor policies via partial denoising,” in *Proceedings of the 42nd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 267, 2025, pp. 8552–8573.
- [17] S. Li, L. Sun, and Y. Chen, “One-step flow policy: Self-distillation for fast visuomotor policies,” *arXiv preprint arXiv:2603.12480*, 2026.
- [18] K. Black, M. Y. Galliker, and S. Levine, “Real-time execution of action chunking flow policies,” in *Advances in Neural Information Processing Systems*, 2025.
- [19] Y. Lu, Z. Liu, X. Fan, Z. Yang, J. Hou, J. Li, K. Ding, and H. Zhao, “Faster: Rethinking real-time flow vlas,” *arXiv preprint arXiv:2603.19199*, 2026.
- [20] Y. Jiang, S. Cheng, Y. Ding, F. Gao, and B. Qi, “Asyncvla: Asynchronous flow matching for vision-language-action models,” *arXiv preprint arXiv:2511.14148*, 2025.
- [21] Y. Chen, S. Zhang, J. Gong, and X. Qiu, “Let it be simple: One-step action generation for vision-language-action models,” *arXiv preprint arXiv:2606.05737*, 2026.
- [22] Y. Zhang, K. Ji, Y. Zou, R. Xu, F. Zheng, and L. Cheng, “Invertible neural network adapter for one-step flow matching in robot manipulation,” *arXiv preprint arXiv:2606.19194*, 2026.
- [23] H. Liang, X. Chen, B. Wang, M. Chen, Y. Liu, Y. Zhang, Z. Chen, T. Yang, Y. Chen, J. Pang, D. Liu, X. Yang, Y. Mu, W. Shao, and P. Luo, “Mm-act: Learn from multimodal parallel generation to act,” *arXiv preprint arXiv:2512.00975*, 2025.
- [24] W. Luan, J. Li, W. Zhao, W. Zhang, T. Wu, and R. Ma, “Snapflow: One-step action generation for flow-matching vlas via progressive self-distillation,” *arXiv preprint arXiv:2604.05656*, 2026.
- [25] Y. Chen, X. Ma, and B. Zhao, “Mean-flow based one-step vision-language-action,” *arXiv preprint arXiv:2603.01469*, 2026.
- [26] Z. Geng, M. Deng, X. Bai, J. Z. Kolter, and K. He, “Mean flows for one-step generative modeling,” in *Advances in Neural Information Processing Systems*, vol. 38, 2025.
- [27] M. Deng, H. Li, T. Li, Y. Du, and K. He, “Generative modeling via drifting,” *arXiv preprint arXiv:2602.04770*, 2026.
- [28] Y. Gao, Y. Shen, S. Zhang, W. Yu, Y. Duan, J. Pan, J. Wu, J. Deng, and Y. Zhang, “Drift-based policy optimization: Native one-step policy learning for online robot control,” in *Proceedings of the 34th ACM International Conference on Multimedia*, 2026, to appear; preprint available at arXiv:2604.03540.
- [29] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, “Learning fine-grained bimanual manipulation with low-cost hardware,” in *Robotics: Science and Systems*, 2023.
- [30] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. Burchfiel, and S. Song, “Diffusion policy: Visuomotor policy learning via action diffusion,” in *Proceedings of Robotics: Science and Systems*, 2023.
- [31] S. Haldar, Z. Peng, and L. Pinto, “Baku: An efficient transformer for multi-task policy learning,” in *Advances in Neural Information Processing Systems*, vol. 37, 2024.
- [32] K. Pertsch, K. Stachowicz, B. Ichter, D. Driess, S. Nair, Q. Vuong, O. Mees, C. Finn, and S. Levine, “FAST: Efficient Action Tokenization for Vision-Language-Action Models,” in *Proceedings of Robotics: Science and Systems*, 2025.
- [33] Y. Ze, G. Zhang, K. Zhang, C. Hu, M. Wang, and H. Xu, “3d diffusion policy: Generalizable visuomotor policy learning via simple 3d representations,” in *Proceedings of Robotics: Science and Systems*, 2024.
- [34] Y. Hu, J.-N. Zaeck, N. Nikolov, Y. Yao, S. Dey, G. Albanese, R. Detry, L. Van Gool, and D. Paudel, “Ar-vla: True autoregressive action expert for vision-language-action models,” *arXiv preprint arXiv:2603.10126*, 2026, accepted to Robotics: Science and Systems 2026.
- [35] K. Frans, D. Hafner, S. Levine, and P. Abbeel, “One step diffusion via shortcut models,” *arXiv preprint arXiv:2410.12557*, 2024.
- [36] Y. Guo, W. Chen, and J. Zhang, “Reactvla: Fast and lightweight reactive robot manipulation via improved mean flow action generation,” *arXiv preprint arXiv:2606.14255*, 2026.
- [37] G. Yan, J. Zhu, Y. Deng, S. Yang, R.-Z. Qiu, X. Cheng, M. Memmel, R. Krishna, A. Goyal, X. Wang, and D. Fox, “Maniflow: A general robot manipulation policy via consistency flow training,” in *Proceedings of The 9th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 305, 2025, pp. 2268–2293.
- [38] J. Sheng, Z. Wang, P. Li, and M. Liu, “Mp1: Meanflow tames policy learning in 1-step for robotic manipulation,” *arXiv preprint arXiv:2507.10543*, 2025.
- [39] H. Fang, Y. Huang, Y. Zhao, P. Weng, X. Li, and Y. Ban, “Omp: One-step meanflow policy with directional alignment,” *arXiv preprint arXiv:2512.19347*, 2025.
- [40] H. Liu, C. Xu, J. Ji, and Y. Mu, “ π_0 -eqm: Equilibrium matching for closed-loop vision-language-action control,” *arXiv preprint arXiv:2605.23128*, 2026.
- [41] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, “Flow matching for generative modeling,” in *International Conference on Learning Representations*, 2023.
- [42] X. Liu, C. Gong, and Q. Liu, “Flow straight and fast: Learning to generate and transfer data with rectified flow,” *arXiv preprint arXiv:2209.03003*, 2022.
- [43] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, “Libero: Benchmarking knowledge transfer for lifelong robot learning,” in *Advances in Neural Information Processing Systems*, 2023.

- [44] T. Chen, Z. Chen, B. Chen, Z. Cai, Y. Liu, Z. Li, Q. Liang, X. Lin, Y. Ge, Z. Gu *et al.*, “Robotwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation,” *arXiv preprint arXiv:2506.18088*, 2025.